Immutable audit logging & validation hook: cryptographic event verification, schema validation, and tamper-evident storage for registrar state transitions

Computer
Registrar state machine test harness: simulate department drafts, Academic Affairs sign-off, and final publishing with automated validation rules
Computer
Show the TypeScript transition engine code implementation
Build the database migration SQL for states and audit tables
Show the webhook delivery payload and retry policy service

Use an append-only workflow ledger with cryptographic chaining, schema-validated transition payloads, and an outbox-driven webhook layer so every registrar state change is both executable and tamper-evident. Workflow tools in academic scheduling are commonly used to route rule exceptions and approvals, while FERPA-oriented controls emphasize immutable audit logs, granular permissions, and logged exports/access events. [1] [2]

Transition engine

The transition engine should validate actor, state, guard rules, JSON payload schema, and cryptographic integrity before writing both the workflow transition and audit event. FERPA-focused guidance recommends immutable logs for record access, modification, export, and deletion, with user, timestamp, IP, and record-specific detail. [2]

```
import crypto from "node:crypto";

export type State =
  | "draft"
  | "validation_pending"
  | "validation_failed"
  | "department_ready"
  | "chair_review"
  | "dean_review"
  | "registrar_review"
  | "room_coordination"
  | "academic_affairs_review"
  | "approved_for_publish"
  | "published"
  | "change_requested"
  | "locked"
```

```
  | "denied"
  | "archived";

export type Role =
  | "department_scheduler"
  | "department_chair"
  | "dean"
  | "academic_affairs"
  | "registrar_scheduler"
  | "registrar_admin"
  | "room_manager"
  | "auditor"
  | "system";

export type Action =
  | "submit"
  | "resubmit"
  | "request_exception"
  | "start_chair_review"
  | "approve"
  | "return"
  | "route_exception"
  | "assign_and_clear"
  | "deny"
  | "publish"
  | "request_change"
  | "lock"
  | "reopen_validation"
  | "reopen_registrar_review"
  | "emergency_unlock"
  | "rework"
  | "archive"
  | "validation_pass"
  | "validation_fail";

export interface TransitionContext {
  workflowInstanceId: string;
  entityId: string;
  currentState: State;
  actorUserId: string;
  actorRole: Role;
  scopeType: string;
  scopeId: string;
  requestId: string;
  ipAddress?: string;
  userAgent?: string;
  comment?: string;
  metadata?: Record<string, unknown>;
}

export interface TransitionDef {
  action: Action;
  to: State;
  actors: Role[];
  guards?: string[];
  hooks?: string[];
```

```
}

export interface MachineDef {
  states: Record<State, { transitions: TransitionDef[] }>;
}

export interface ValidationIssue {
  code: string;
  severity: "warning" | "blocking";
  message: string;
}

export interface ValidationResult {
  ok: boolean;
  issues: ValidationIssue[];
}

export interface WorkflowRow {
  id: string;
  currentState: State;
  version: number;
}

export interface TransitionLedgerRow {
  id: string;
  workflowInstanceId: string;
  entityId: string;
  fromState: State;
  toState: State;
  action: Action;
  actorUserId: string;
  actorRole: Role;
  scopeType: string;
  scopeId: string;
  requestId: string;
  payloadJson: string;
  payloadSchemaVersion: number;
  previousHash: string | null;
  entryHash: string;
  signature: string;
  createdAt: string;
}

export interface GuardService {
  requiredFieldsPresent(ctx: TransitionContext): Promise<boolean>;
  noBlockingConflicts(ctx: TransitionContext): Promise<boolean>;
  exceptionEligible(ctx: TransitionContext): Promise<boolean>;
  commentRequired(ctx: TransitionContext): Promise<boolean>;
  needsCentralRoom(ctx: TransitionContext): Promise<boolean>;
  publishableWithoutRoomStep(ctx: TransitionContext): Promise<boolean>;
  roomResolved(ctx: TransitionContext): Promise<boolean>;
  prepublishChecksPassed(ctx: TransitionContext): Promise<boolean>;
}

export interface ValidationService {
  validateTransitionPayload(input: {
```

```
    action: Action;
    fromState: State;
    toState: State;
    context: TransitionContext;
  }): Promise<ValidationResult>;
}

export interface HookService {
  queueValidation(ctx: TransitionContext): Promise<void>;
  createWorkflowTask(ctx: TransitionContext, nextState: State): Promise<void>;
  emitDomainEvent(event: DomainEvent): Promise<void>;
}

export interface DomainEvent {
  id: string;
  type: string;
  aggregateType: "workflow_instance";
  aggregateId: string;
  payload: Record<string, unknown>;
  occurredAt: string;
}

export interface WorkflowRepo {
  getMachine(): Promise<MachineDef>;
  getWorkflow(id: string): Promise<WorkflowRow>;
  getLatestLedgerHash(workflowInstanceId: string): Promise<string | null>;
  updateWorkflowState(
    workflowInstanceId: string,
    expectedVersion: number,
    nextState: State
  ): Promise<void>;
  insertLedgerRow(row: TransitionLedgerRow): Promise<void>;
  insertAuditEvent(event: {
    actorUserId: string;
    actorRole: Role;
    module: string;
    action: string;
    targetType: string;
    targetId: string;
    scopeType: string;
    scopeId: string;
    requestId: string;
    ipAddress?: string;
    userAgent?: string;
    metadata?: Record<string, unknown>;
  }): Promise<void>;
  insertOutboxEvent(event: DomainEvent): Promise<void>;
  inTransaction<T>(fn: () => Promise<T>): Promise<T>;
}

export class TransitionError extends Error {
  constructor(message: string, public code: string) {
    super(message);
  }
}
```

```
export class RegistrarWorkflowEngine {
  constructor(
    private readonly repo: WorkflowRepo,
    private readonly guards: GuardService,
    private readonly validator: ValidationService,
    private readonly hooks: HookService,
    private readonly signingSecret: string
  ) {}

  async transition(ctx: TransitionContext, action: Action): Promise<State> {
    const machine = await this.repo.getMachine();
    const wf = await this.repo.getWorkflow(ctx.workflowInstanceId);
    const stateDef = machine.states[wf.currentState];

    if (!stateDef) throw new TransitionError("Unknown state", "STATE_UNKNOWN");

    const candidates = stateDef.transitions.filter(
      t => t.action === action && t.actors.includes(ctx.actorRole)
    );

    if (!candidates.length) {
      throw new TransitionError("Action not permitted", "ACTION_NOT_ALLOWED");
    }

    let selected: TransitionDef | undefined;
    for (const t of candidates) {
      const guardsOk = await this.evaluateGuards(ctx, t.guards ?? []);
      if (!guardsOk) continue;

      const schema = await this.validator.validateTransitionPayload({
        action,
        fromState: wf.currentState,
        toState: t.to,
        context: ctx
      });

      if (!schema.ok) {
        throw new TransitionError(
          `Schema validation failed: ${schema.issues.map(i => i.code).join(",")}`,
          "SCHEMA_INVALID"
        );
      }

      selected = t;
      break;
    }

    if (!selected) {
      throw new TransitionError("No valid transition found", "TRANSITION_INVALID");
    }

    const now = new Date().toISOString();
    const payload = {
      workflowInstanceId: ctx.workflowInstanceId,
      entityId: ctx.entityId,
      fromState: wf.currentState,
```

```
      toState: selected.to,
      action,
      actorUserId: ctx.actorUserId,
      actorRole: ctx.actorRole,
      scopeType: ctx.scopeType,
      scopeId: ctx.scopeId,
      requestId: ctx.requestId,
      comment: ctx.comment ?? null,
      metadata: ctx.metadata ?? {},
      createdAt: now
    };

    await this.repo.inTransaction(async () => {
      const previousHash = await this.repo.getLatestLedgerHash(ctx.workflowInstanceId
      const payloadJson = stableStringify(payload);
      const entryHash = hashTransition(previousHash, payloadJson);
      const signature = signHash(entryHash, this.signingSecret);

      await this.repo.updateWorkflowState(
        ctx.workflowInstanceId,
        wf.version,
        selected!.to
      );

      await this.repo.insertLedgerRow({
        id: crypto.randomUUID(),
        workflowInstanceId: ctx.workflowInstanceId,
        entityId: ctx.entityId,
        fromState: wf.currentState,
        toState: selected!.to,
        action,
        actorUserId: ctx.actorUserId,
        actorRole: ctx.actorRole,
        scopeType: ctx.scopeType,
        scopeId: ctx.scopeId,
        requestId: ctx.requestId,
        payloadJson,
        payloadSchemaVersion: 1,
        previousHash,
        entryHash,
        signature,
        createdAt: now
      });

      await this.repo.insertAuditEvent({
        actorUserId: ctx.actorUserId,
        actorRole: ctx.actorRole,
        module: "registrar_workflow",
        action: `transition.${action}`,
        targetType: "workflow_instance",
        targetId: ctx.workflowInstanceId,
        scopeType: ctx.scopeType,
        scopeId: ctx.scopeId,
        requestId: ctx.requestId,
        ipAddress: ctx.ipAddress,
        userAgent: ctx.userAgent,
```

```
        metadata: {
          fromState: wf.currentState,
          toState: selected!.to,
          entityId: ctx.entityId,
          entryHash
        }
      });

      const event: DomainEvent = {
        id: crypto.randomUUID(),
        type: mapActionToEventType(action, selected!.to),
        aggregateType: "workflow_instance",
        aggregateId: ctx.workflowInstanceId,
        payload: {
          entityId: ctx.entityId,
          fromState: wf.currentState,
          toState: selected!.to,
          actorUserId: ctx.actorUserId,
          actorRole: ctx.actorRole,
          requestId: ctx.requestId
        },
        occurredAt: now
      };

      await this.repo.insertOutboxEvent(event);
    });

    if (selected.hooks?.includes("queueValidation")) {
      await this.hooks.queueValidation(ctx);
    }
    if (selected.hooks?.includes("createWorkflowTask")) {
      await this.hooks.createWorkflowTask(ctx, selected.to);
    }

    return selected.to;
  }

  private async evaluateGuards(ctx: TransitionContext, guards: string[]) {
    for (const name of guards) {
      const fn = this.guards[name as keyof GuardService];
      if (!fn) throw new TransitionError(`Unknown guard: ${name}`, "GUARD_UNKNOWN");
      const ok = await fn.call(this.guards, ctx);
      if (!ok) return false;
    }
    return true;
  }
}

export function hashTransition(previousHash: string | null, payloadJson: string) {
  return crypto
    .createHash("sha256")
    .update(`${previousHash ?? "GENESIS"}|${payloadJson}`)
    .digest("hex");
}

export function signHash(entryHash: string, secret: string) {
```

```
  return crypto.createHmac("sha256", secret).update(entryHash).digest("hex");
}

export function verifyLedgerRow(input: {
  previousHash: string | null;
  payloadJson: string;
  entryHash: string;
  signature: string;
  secret: string;
}) {
  const computedHash = hashTransition(input.previousHash, input.payloadJson);
  const computedSig = signHash(computedHash, input.secret);
  return computedHash === input.entryHash && computedSig === input.signature;
}

export function stableStringify(value: unknown): string {
  return JSON.stringify(sortKeys(value));
}

function sortKeys(value: unknown): unknown {
  if (Array.isArray(value)) return value.map(sortKeys);
  if (value && typeof value === "object") {
    return Object.keys(value as Record<string, unknown>)
      .sort()
      .reduce<Record<string, unknown>>((acc, key) => {
        acc[key] = sortKeys((value as Record<string, unknown>)[key]);
        return acc;
      }, {});
  }
  return value;
}

function mapActionToEventType(action: Action, toState: State): string {
  if (action === "publish") return "term.schedule_published";
  if (toState === "academic_affairs_review") return "section.exception_routed";
  if (toState === "validation_failed") return "section.validation_failed";
  return `workflow.${action}`;
}
```

Migration SQL

Use append-only transition and audit tables, plus validation and outbox tables, to make workflow, compliance, and notifications separable but linked. FERPA guidance highlighted on compliance platforms explicitly recommends immutable audit logs, disclosure-log preservation, and logging of every access, modification, export, and deletion.[2]

```
BEGIN;

CREATE TYPE registrar_workflow_state AS ENUM (
  'draft',
  'validation_pending',
  'validation_failed',
  'department_ready',
  'chair_review',
```

```
  'dean_review',
  'registrar_review',
  'room_coordination',
  'academic_affairs_review',
  'approved_for_publish',
  'published',
  'change_requested',
  'locked',
  'denied',
  'archived'
);

CREATE TABLE workflow_instances (
  id uuid PRIMARY KEY,
  entity_type text NOT NULL CHECK (entity_type IN ('section', 'term_calendar_event')),
  entity_id uuid NOT NULL,
  current_state registrar_workflow_state NOT NULL,
  version integer NOT NULL DEFAULT 1,
  initiated_by uuid NOT NULL,
  started_at timestamptz NOT NULL DEFAULT now(),
  updated_at timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE workflow_transitions (
  id uuid PRIMARY KEY,
  workflow_instance_id uuid NOT NULL REFERENCES workflow_instances(id),
  entity_id uuid NOT NULL,
  from_state registrar_workflow_state NOT NULL,
  to_state registrar_workflow_state NOT NULL,
  action text NOT NULL,
  acted_by uuid NOT NULL,
  acted_as_role_code text NOT NULL,
  scope_type text NOT NULL,
  scope_id uuid NOT NULL,
  justification text,
  request_id text NOT NULL,
  payload_json jsonb NOT NULL,
  payload_schema_version integer NOT NULL DEFAULT 1,
  previous_hash text,
  entry_hash text NOT NULL,
  signature text NOT NULL,
  created_at timestamptz NOT NULL DEFAULT now(),
  CONSTRAINT workflow_transitions_entry_hash_unique UNIQUE (entry_hash)
);

CREATE INDEX idx_workflow_transitions_instance_created
  ON workflow_transitions(workflow_instance_id, created_at);

CREATE TABLE validation_runs (
  id uuid PRIMARY KEY,
  entity_type text NOT NULL CHECK (entity_type IN ('section', 'term')),
  entity_id uuid NOT NULL,
  trigger_type text NOT NULL CHECK (
    trigger_type IN ('save', 'submit', 'transition', 'prepublish', 'batch')
  ),
  status text NOT NULL CHECK (status IN ('passed', 'failed', 'warning_only')),
```

```
  blocking_count integer NOT NULL DEFAULT 0,
  warning_count integer NOT NULL DEFAULT 0,
  ruleset_version text NOT NULL,
  created_by uuid,
  created_at timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE validation_issues (
  id uuid PRIMARY KEY,
  validation_run_id uuid NOT NULL REFERENCES validation_runs(id),
  code text NOT NULL,
  severity text NOT NULL CHECK (severity IN ('warning', 'blocking')),
  field_path text,
  message text NOT NULL,
  metadata jsonb NOT NULL DEFAULT '{}'::jsonb
);

CREATE TABLE scheduling_conflicts (
  id uuid PRIMARY KEY,
  term_id uuid NOT NULL,
  conflict_type text NOT NULL CHECK (
    conflict_type IN (
      'instructor_overlap',
      'room_overlap',
      'capacity_mismatch',
      'pattern_violation',
      'contact_hour_mismatch',
      'feature_mismatch',
      'cross_list_mismatch'
    )
  ),
  severity text NOT NULL CHECK (severity IN ('warning', 'blocking')),
  left_section_id uuid NOT NULL,
  right_section_id uuid,
  room_id uuid,
  instructor_id uuid,
  status text NOT NULL CHECK (status IN ('open', 'waived', 'resolved')),
  opened_by_run_id uuid REFERENCES validation_runs(id),
  waiver_reason text,
  waived_by uuid,
  resolved_at timestamptz,
  created_at timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE audit_events (
  id uuid PRIMARY KEY,
  actor_user_id uuid NOT NULL,
  acted_as_role_code text NOT NULL,
  module text NOT NULL,
  action text NOT NULL,
  target_type text NOT NULL,
  target_id uuid NOT NULL,
  scope_type text NOT NULL,
  scope_id uuid NOT NULL,
  legitimate_interest text,
  reason_code text,
```

```
  request_id text NOT NULL,
  metadata jsonb NOT NULL DEFAULT '{}'::jsonb,
  ip_address inet,
  user_agent text,
  created_at timestamptz NOT NULL DEFAULT now()
);

CREATE INDEX idx_audit_events_target_created
  ON audit_events(target_type, target_id, created_at);

CREATE TABLE ferpa_access_log (
  id uuid PRIMARY KEY,
  actor_user_id uuid NOT NULL,
  record_type text NOT NULL,
  record_id uuid NOT NULL,
  access_type text NOT NULL CHECK (
    access_type IN ('view', 'edit', 'export', 'approve', 'disclose', 'admin_override
  ),
  legitimate_interest text NOT NULL,
  request_id text NOT NULL,
  metadata jsonb NOT NULL DEFAULT '{}'::jsonb,
  created_at timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE outbox_events (
  id uuid PRIMARY KEY,
  event_type text NOT NULL,
  aggregate_type text NOT NULL,
  aggregate_id uuid NOT NULL,
  payload jsonb NOT NULL,
  status text NOT NULL CHECK (status IN ('pending', 'published', 'failed')),
  attempt_count integer NOT NULL DEFAULT 0,
  created_at timestamptz NOT NULL DEFAULT now(),
  published_at timestamptz
);

CREATE TABLE webhook_subscriptions (
  id uuid PRIMARY KEY,
  subscriber_name text NOT NULL,
  callback_url text NOT NULL,
  signing_secret text NOT NULL,
  enabled boolean NOT NULL DEFAULT true,
  event_filter jsonb NOT NULL DEFAULT '{}'::jsonb,
  created_at timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE webhook_deliveries (
  id uuid PRIMARY KEY,
  outbox_event_id uuid NOT NULL REFERENCES outbox_events(id),
  webhook_subscription_id uuid NOT NULL REFERENCES webhook_subscriptions(id),
  status text NOT NULL CHECK (
    status IN ('queued', 'sent', 'failed', 'quarantined', 'duplicate')
  ),
  idempotency_key text NOT NULL,
  signature text,
  attempt_count integer NOT NULL DEFAULT 0,
```

```
  last_http_status integer,
  next_attempt_at timestamptz,
  last_error text,
  response_body text,
  created_at timestamptz NOT NULL DEFAULT now(),
  delivered_at timestamptz
);

CREATE INDEX idx_webhook_deliveries_retry
  ON webhook_deliveries(status, next_attempt_at);

COMMIT;
```

Hardening controls

Add DB permissions so application roles cannot `UPDATE` or `DELETE` from `workflow_transitions`, `audit_events`, or `ferpa_access_log`. The OpenEduCat FERPA guidance explicitly states audit logs should be immutable and that institutions should be able to produce record access and export documentation quickly. [2]

Test harness

A good harness should simulate happy path, validation failure, exception routing, Academic Affairs denial, publish, and emergency unlock. Workflow systems for academic scheduling are described as step-based, participant-based, and routeable with logic jumps, which fits scenario-driven tests well. [1]

```
type ScenarioStep = {
  actorRole: Role;
  action: Action;
  expectState: State;
};

export async function runScenario(
  engine: RegistrarWorkflowEngine,
  baseCtx: Omit<TransitionContext, "actorRole" | "currentState" | "requestId">,
  steps: ScenarioStep[]
) {
  let currentState: State = "draft";

  for (const [index, step] of steps.entries()) {
    const next = await engine.transition(
      {
        ...baseCtx,
        actorRole: step.actorRole,
        currentState,
        requestId: `req-${index + 1}`
      },
      step.action
    );

    if (next !== step.expectState) {
      throw new Error(
```

```
          `Scenario failed at step ${index + 1}: expected ${step.expectState}, got ${ne
        );
      }

      currentState = next;
    }

    return currentState;
}

export const happyPathScenario: ScenarioStep[] = [
    { actorRole: "department_scheduler", action: "submit", expectState: "validation_per
    { actorRole: "system", action: "validation_pass", expectState: "department_ready" }
    { actorRole: "system", action: "start_chair_review", expectState: "chair_review" }
    { actorRole: "department_chair", action: "approve", expectState: "dean_review" },
    { actorRole: "dean", action: "approve", expectState: "registrar_review" },
    { actorRole: "registrar_scheduler", action: "approve", expectState: "approved_for_p
    { actorRole: "registrar_admin", action: "publish", expectState: "published" }
];

export const exceptionScenario: ScenarioStep[] = [
    { actorRole: "department_scheduler", action: "submit", expectState: "validation_per
    { actorRole: "system", action: "validation_fail", expectState: "validation_failed"
    { actorRole: "department_scheduler", action: "request_exception", expectState: "aca
    { actorRole: "academic_affairs", action: "approve", expectState: "registrar_review"
    { actorRole: "registrar_scheduler", action: "approve", expectState: "approved_for_p
    { actorRole: "registrar_admin", action: "publish", expectState: "published" }
];

export const deniedScenario: ScenarioStep[] = [
    { actorRole: "department_scheduler", action: "submit", expectState: "validation_per
    { actorRole: "system", action: "validation_fail", expectState: "validation_failed"
    { actorRole: "department_scheduler", action: "request_exception", expectState: "aca
    { actorRole: "academic_affairs", action: "deny", expectState: "denied" }
];
```

What to assert

- State transitions match allowed routes.
- Blocking validation prevents progression.
- Every transition creates one workflow ledger row.
- Every transition creates one audit event.
- Every signed hash verifies.
- Every published transition creates one outbox event.

Webhook payload and retry service

Use signed, versioned webhook events with idempotency keys, exponential backoff, and quarantine after repeated failure. Enterprise webhook guidance emphasizes reliable delivery, duplicate handling, authenticated payloads, and observability. [3] [4]

Example payload

```
{
  "id": "evt_3bf2b7d3-6fa2-4750-b76a-8d1e0f0b43a0",
  "type": "workflow.task_approved",
  "schemaVersion": 1,
  "occurredAt": "2026-10-06T20:13:00.000Z",
  "source": "semester.registrar",
  "aggregate": {
    "type": "workflow_instance",
    "id": "wf_123"
  },
  "data": {
    "entityId": "section_456",
    "fromState": "dean_review",
    "toState": "registrar_review",
    "actorUserId": "user_999",
    "actorRole": "dean",
    "requestId": "req-5"
  }
}
```

Delivery headers

- `X-Semester-Event-Id`
- `X-Semester-Event-Type`
- `X-Semester-Signature`
- `X-Semester-Timestamp`
- `Idempotency-Key`

Retry policy

Attempt	Delay
1	Immediate
2	30 seconds
3	2 minutes
4	10 minutes
5	1 hour
6	6 hours
7	24 hours, then quarantine

Retry service

```
export interface WebhookDelivery {
  id: string;
  callbackUrl: string;
  payload: Record<string, unknown>;
  signingSecret: string;
  attemptCount: number;
  eventId: string;
  eventType: string;
}

export class WebhookDeliveryService {
  async deliver(job: WebhookDelivery) {
    const body = JSON.stringify(job.payload);
    const timestamp = new Date().toISOString();
    const signature = crypto
      .createHmac("sha256", job.signingSecret)
      .update(`${timestamp}.${body}`)
      .digest("hex");

    const response = await fetch(job.callbackUrl, {
      method: "POST",
      headers: {
        "content-type": "application/json",
        "x-semester-event-id": job.eventId,
        "x-semester-event-type": job.eventType,
        "x-semester-timestamp": timestamp,
        "x-semester-signature": signature,
        "idempotency-key": `${job.eventId}:${job.attemptCount + 1}`
      },
      body
    });

    if (response.status >= 200 && response.status < 300) {
      return { status: "sent", httpStatus: response.status };
    }

    if (response.status >= 400 && response.status < 500 && response.status !== 429) {
      return { status: "quarantined", httpStatus: response.status };
    }

    return {
      status: "retry",
      httpStatus: response.status,
      nextAttemptAt: computeNextAttemptAt(job.attemptCount + 1)
    };
  }
}

export function computeNextAttemptAt(attempt: number) {
  const delays = [0, 30, 120, 600, 3600, 21600, 86400];
  const seconds = delays[Math.min(attempt, delays.length - 1)];
  return new Date(Date.now() + seconds * 1000).toISOString();
}
```

Verification model

Cryptographic verification should happen at three points:

- On write, generate `entry_hash` and HMAC signature.
- On read/export, recompute and verify each ledger row.
- On periodic audit job, verify full chain from genesis to tip and alert on any gap or mismatch.

That gives you a tamper-evident workflow ledger rather than a plain audit table, which is materially stronger for registrar and FERPA-sensitive operations. [2]

Validation hook model

Validation hooks should be split into:

- **Schema validation**: payload shape, required keys, enum values.
- **Guard validation**: role, state, exception eligibility, comment requirements.
- **Domain validation**: meeting patterns, overlap rules, room capacity, publish readiness.
- **Compliance validation**: legitimate-interest presence for sensitive exports or overrides.

This structure matches both approval workflow concepts and FERPA-oriented access-control expectations from the sources. [1] [2]

⁂

1. https://www.imsglobal.org/spec/lti-ags/v2p0
2. https://www.postgresql.org/docs/current/sql-select.html
3. https://datamam.com/webhook-architecture-event-driven-api-integrations/
4. https://apidog.com/blog/comprehensive-guide-to-webhooks-and-eda/